Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Dynamic Insertion	add_item()	$O(1)$	$O(1)$ amortized	$O(n)$	$O(n)$	When currentCount < maxCapacity, insertion directly executes dataArray[currentCount] = val, requiring constant time. When capacity is full, a new array is allocated and up to n existing elements are copied, giving $O(n)$ worst-case time. Across many insertions, resizing by doubling gives an amortized $O(1)$ insertion cost.
Dynamic Resize	Resize section inside add_item()	$O(n)$	$O(n)$ when resize occurs	$O(n)$	$O(n)$	A new array of size $2C$ is allocated and the existing C elements are copied. Therefore, the copy operation performs approximately C assignments: $T(n) = C = O(n)$. Two arrays temporarily exist,

						so additional memory is $O(n)$.
Linear Deletion	remove_item_at()	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Removing the final element requires no shifting, so it takes $O(1)$. Removing an element near the beginning requires shifting approximately $n - i - 1$ elements. The redundant nested loop does not change the asymptotic bound because the inner loop executes only once due to break.
Linear Search	findItem()	$O(1)$	$O(n)$	$O(n)$	$O(1)$	If the target is the first element, only one comparison is needed. If the target is near the middle, approximately $n/2$ comparisons are required. If the target is at the last position or absent, n comparisons are required. Therefore, worst case is $O(n)$.

Array Display	printAll()	$O(n)$	$O(n)$	$O(n)$	$O(1)$	The function traverses the array and outputs each element. For n elements, approximately n iterations are performed: $T(n) = n$, resulting in $O(n)$ time. The function does not allocate additional storage, so auxiliary space is $O(1)$.
Matrix Transpose	processMatrix()	$O(R \times$	$O(R \times C)$	$O(R \times$	$O(R \times C)$	Every element of an $R \times C$ matrix is visited exactly once. The nested loops perform $R \times C$ assignments: $T(R, C) = R \times C$, giving $O(RC)$ time. The additional transpose matrix t requires $R \times C$ storage.
Index Validation	remove_item_at() / bounds checking	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Checking conditions such as idx < 0 or idx >= currentCount requires only a constant number of comparisons, independent of n.

Memory Cleanup	delete[] dataArray	$O(n)$ *	$O(n)$ *	$O(n)$ *	$O(1)$ auxiliary	Releasing an allocated array may require the runtime to process the allocated block. For an array of trivial int values, the exact implementation cost is runtime-dependent; for algorithmic analysis, cleanup can be treated as $O(n)$ in the general resource-lifecycle model.